HDOC Day-62 Activity

Merging Two Sorted Arrays Using Pointers and the Two-Pointer Technique

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-62

Aim

To write a C program that merges two already sorted integer arrays into a third sorted array using the **two-pointer technique**. Both input arrays are passed to **merge_sorted_arrays()** using their addresses and accessed through pointers.

Problem Requirements

- Read the sizes and elements of two sorted variable-length arrays, **arr1[]** and **arr2[]**.
- Pass both arrays to **merge_sorted_arrays()** using their addresses.
- Merge both arrays into a third array named **merged[]**.
- Keep all elements in ascending order.
- Preserve duplicate elements.
- Display the merged elements separated by spaces.
- Use the efficient two-pointer technique.

Two-Pointer Idea

Because both input arrays are already sorted, we do not need to sort the combined data again. Pointer indices **i** and **j** start at the beginning of arr1 and arr2. At each step, the smaller current element is copied into merged[], and only the pointer belonging to that element moves forward.

Algorithm

1. Read n1 and the n1 sorted elements of arr1[].
2. Read n2 and the n2 sorted elements of arr2[].
3. Pass both array addresses and sizes to **merge_sorted_arrays(arr1, n1, arr2, n2)**.
4. Create merged[n1+n2] and initialize i = 0, j = 0, k = 0.
5. While both arrays still contain unprocessed elements, compare ***(arr1+i)** and ***(arr2+j)**.
6. Copy the smaller value into merged[k] and advance the corresponding pointer. Using <= preserves equal values instead of removing duplicates.
7. Copy any remaining elements from arr1, then any remaining elements from arr2.
8. Print merged[] separated by spaces.

C Program

```
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2)
{
    int merged[n1 + n2];
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2)
    {
        if (*(arr1 + i) <= *(arr2 + j))
        {
            merged[k] = *(arr1 + i);
            i++;
        }
        else
        {
            merged[k] = *(arr2 + j);
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        merged[k] = *(arr1 + i);
        i++;
        k++;
    }

    while (j < n2)
    {
        merged[k] = *(arr2 + j);
        j++;
        k++;
    }

    for (int x = 0; x < n1 + n2; x++)
    {
        printf("%d", merged[x]);

        if (x < n1 + n2 - 1)
            printf(" ");
    }
}

int main()
{
    int n1, n2;

    printf("Enter size of first array: ");
    scanf("%d", &n1);
    int arr1[n1];

    printf("Enter elements of first array: ");
    for (int i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter size of second array: ");
    scanf("%d", &n2);
    int arr2[n2];

    printf("Enter elements of second array: ");
    for (int i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    merge_sorted_arrays(arr1, n1, arr2, n2);

    return 0;
}
```

Pointer Usage

arr1 and **arr2** are passed as addresses. The function parameters **int *arr1** and **int *arr2** receive them. Expressions ***(arr1+i)** and ***(arr2+j)** access the current values while the two pointers move through the arrays.

Example 1 Dry Run

arr1 value	arr2 value	Selected	Merged so far
1	2	1	1
3	2	2	1 2
3	4	3	1 2 3
5	4	4	1 2 3 4
5	6	5	1 2 3 4 5
7	6	6	1 2 3 4 5 6
7	8	7	1 2 3 4 5 6 7
-	8	8	1 2 3 4 5 6 7 8

Test Cases / Expected Results

Case	Array 1	Array 2	Expected Output
Example 1	1 3 5 7	2 4 6 8	1 2 3 4 5 6 7 8
Example 2	1 2 2 4	2 3 5	1 2 2 2 3 4 5
Example 3	1 4 7	2 3 5 6 8	1 2 3 4 5 6 7 8

Duplicate Handling - Example 2

Array 1 contains two copies of 2 and Array 2 contains another 2. The merge operation does not remove equal values, so all three copies appear in the result: **1 2 2 2 3 4 5**.

Why the Output Remains Sorted

At every step, the smaller of the two currently available values is selected. Since each input array is already sorted, no smaller unprocessed value can exist behind either pointer. Therefore, merged[] remains sorted throughout the operation.

Complexity

Time Complexity: O(n1 + n2), because each input element is visited and copied exactly once.

Space Complexity: O(n1 + n2) for the required merged[] output array.

Result

The program successfully merges two sorted arrays into one ascending array using pointer-based access and the two-pointer technique. Duplicate elements are preserved and the merge runs in linear time.

Execution Example

```
Enter size of first array: 4
Enter elements of first array: 1 3 5 7
Enter size of second array: 4
Enter elements of second array: 2 4 6 8
1 2 3 4 5 6 7 8
```